

Recipe App

CSC-337 Final Project Report

- **Motivation**

Our recipe idea began by asking us: how can we create a community tool?. A place where anyone can contribute a recipe, browse what others have made, and leave feedback. This gave us the opportunity to build this platform around user interaction and shared content.

Beyond the idea, the project gave us a meaningful way to bring together everything learned throughout the semester. Building a platform with real accounts, persistent data, and live interaction between users and content. Trying to make an idea into a working product. We wanted the end result to be similar to a social hub for recipe enthusiasts.

- **Modules Overview**

The platform is structured around three modules, each managing its own data and responsibilities, while remaining connected through a shared server and session. This separation keeps the codebase organized and maintainable.

User Module

The user module is the entry point of the entire application. It handles registration, login, and logout, this module interacts with MongoDB. Each account stores a name, email, occupation, and a bcrypt-hashed password. When a user successfully authenticates, a server-side session is created that carries their identity across all other pages. It acts as the gatekeeper for the entire platform. The user also stores the recipe IDs for their favorite recipes.

Recipe Module

It presents all submitted recipes as a browsable DB, with each card showing the recipe image, title, and a brief description. Clicking a card opens a full detail view that includes prep time, cook time, total time, serving time, and step-by-step instructions. By clicking on the heart, you are also able to favorite recipes which are then displayed in your home tab. Submitted recipes are stored persistently in MongoDB.

How the Modules Connect

The three modules are connected together through a shared session. A user cannot reach the recipe list or leave a comment without first being authenticated. Once logged in, their session persists as the user moves between the recipe browser, the detail view, and the comments page. The flow is linear: authenticate, browse, read, engage, and leave.

- **Functionalities**

Here we can see the core workflow of the application, covering both user interaction and the underlying data operations that support them.

Account Management

1. New users can register with a name, email, occupation, and a password.
2. Existing users log in with email and password. Incorrect credentials produce an error message.
3. Logging out destroys the session and returns the user to the login screen.
4. Passwords are hashed with bcrypt before being saved. Plain-text passwords are never stored.

Recipe Browsing and Creation

1. The recipes page loads all submissions dynamically from the server and renders them as interactive cards.
2. Each card links to a dedicated page that displays the full recipe with a structured ingredients list and numbered instructions.

3. The creation form accepts a title, prep time, cook time, servings, ingredients, instructions, and an image URL. Total time is calculated server-side.
4. Recipes are also able to be favorite'd which are stored to the user based on ID and can be seen at the home page.
5. Recipes also are able to be edited only by the user who published it.
6. For not logged in users, they are able to create recipes and browse under the username anonymous.

Reviews and Ratings

1. Users can submit a written comment and a 1 to 5 star rating for any recipe.
2. The average rating across all reviews is displayed at the top of the comments section.
3. Users can delete their comments. Server verifies ownership before allowing deletion.
4. Reviews can also be seen on the recipe cards.
5. Not logged in users cannot leave reviews

● Technical Details

Tech Stack

The backend is built with [Node.js](#) and **Express**, running on port 8080. Express handles all routing, serves static frontend files from public directory, and parses both **JSON** and **URL**-encoded request bodies. The **express-session** middleware manages server-side sessions. MongoDB is used via the Mongoose ODM for user, recipe, and comment data. On the frontend, all pages are built with plain **HTML**, **CSS**, and **JavaScript** using 'fetch()' calls to communicate with the backend API.

Data Storage

Three storage strategies are used intentionally across the three modules. User accounts require structured querying (looking up by email), so MongoDB with a Mongoose schema was the right choice. The user schema defines five fields: name, email, occupation, password, and favorites. Recipes are stored in MongoDB with a structure holding ingredients, steps, time for the recipe, a recipeID and a url for an

image. Comments has a structure that stores the user, comment, rating and recipeID. All three modules use Mongoose schemas for consistent data access. .

Session Handling

Sessions are created server-side at the moment a user successfully registers or logs in. The session stores the user's database ID and display name. A cookie is sent to the browser to maintain the session across page navigation. Every protected endpoint checks for 'req.session.userId' before responding — if it is absent the server returns a 401 status and the frontend redirects the user to the login page. Logout calls 'req.session.destroy()' and clears the cookie, making the re-authentication necessary for any further access.

API Structure

The backend exposes three groups of **REST-style** endpoints:

1. **/api/users** – register, login, logout, and profile retrieval.
2. **/api/recipes** – fetch all recipes and submit a new one.
3. **/api/comments** – fetch comments for a recipe, post a new review, and delete a comment.

All three route groups are mounted on a single Express server, allowing the entire application to run as one unified process accessible at <http://localhost:8080>.